

SMS-LLM: A Similarity-Based Multi-Objective Scheduling Algorithm for Large Language Model

Submitted in fulfilment of the requirements for the degree of

Bachelor of Technology by

Vikram (22CSB0B32)

Tanmay Soni (22CSB0B25)

Supervisor:

Sanjaya Kumar Panda

Assistant Professor, Department of CSE



Department of Computer Science and Engineering

National Institute of Technology, Warangal

India

April 2026

Acknowledgement

I would like to express my heartily gratitude towards **Prof. Sanjaya Kumar Panda** for his valuable guidance, supervision, suggestions, encouragement and the help throughout the semester, for the completion of my project work. He kept me going when I was down and gave me the courage to keep moving forward.

I also want to thank evaluation committee for their valuable suggestions and conducting smooth presentation of the project.

Vikram (22CSB0B32)

Tanmay Soni (22CSB0B25)

Declaration

I declare that this written submission represents my ideas, my supervisor's ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Vikram
22CSB0B32

Tanmay Soni
22CSB0B25

APPROVAL SHEET

This Project Work entitled "**SMS-LLM: A Similarity-Based Multi-Objective Scheduling Algorithm for Large Language Model** by **Vikram (22CSB0B32)** and **Tanmay Soni (22CSB0B25)** is approved for

the degree of Bachelor of Technology (B.Tech).

Examiners

Prof. Sriram Kailasam, Prof. Shivadarshan S L and Prof. Manoj Kumar Patra

Supervisor

Assistant Professor Sanjaya Kumar Panda

Head CSE (HoD)

Prof. Rashmi Ranjan Rout

NIT Warangal

Certificate

This is to certify that the Project work entitled "**SMS-LLM: A Similarity-Based Multi-Objective Scheduling Algorithm for Large Language Model**" is a bonafide record of work carried out by "**Vikram (22CSB0B32) and Tanmay Soni (22CSB0B25)**", submitted to Prof. Sanjaya Kumar Panda of Department of Computer Science and Engineering, in partial fulfilment of the requirements for the award of the degree of B.Tech at "National Institute of Technology, Warangal" during the 2022-2026.

(Supervisor Signature)

Sanjaya Kumar Panda

Assistant Professor

Department of Computer Science and Engineering

Abstract

The increasing availability of Large Language models (LLMs) introduces the challenge of selecting the most suitable LLM for a given query (prompt), as different LLMs exhibit varying performance characteristics such as response latency, output quality, token consumption, and computational cost depending on the input prompt. Static single-model deployment fails to account for this variability, leading to suboptimal performance. This paper proposes SMS-LLM, a similarity-based multi-objective scheduling algorithm for adaptive multi-LLM selection. SMS-LLM constructs a knowledge base containing prompt embeddings along with per-LLM performance metrics, including latency, quality score, and token count, where quality is estimated using an LLM-as-a-Judge mechanism. For incoming prompts, semantic similarity is used to identify relevant historical prompts, and their performance metrics are used to estimate each LLM’s expected behavior. Based on these estimates, SMS-LLM selects the most suitable model using latency-oriented (LS-LLM), quality-oriented (QS-LLM), or weighted multi-objective scheduling. Experiments are conducted using LLaMA3, Mistral, and Qwen2.5 across dataset sizes of 500, 1000, and 5000 prompts. Results show that LS-LLM reduces latency by up to 29.1%, QS-LLM improves quality score up to 5.76%, and SMS-LLM achieves balanced performance with latency reduction up to 14.20%, quality improvement up to 1.6%, and token count reduction up to 25.30% compared to static and random baseline algorithms. These results demonstrate that similarity-based scheduling enables efficient and scalable multi-objective LLM selection without requiring supervised training or reinforcement learning.

Contents

Acknowledgement	ii
Declaration	iii
Approval	i
Certificate	ii
Abstract	iii
1 Introduction	1
2 Background	3
2.1 Large Language Models	3
2.2 Prompt Embeddings and Semantic Similarity	3
2.3 Performance Metrics for LLMs	4
2.4 LLM-as-a-Judge	4
2.5 LLM Scheduling and Model Selection	4
3 Related Work	5
4 Proposed Solution	6
4.0.1 Offline Phase: Knowledge Base Construction	6
4.0.2 Online Phase: Similarity-Based Query Scheduling	7
5 Simulation	10
5.1 Simulation Setup	10
5.2 Baseline Algorithms	11
5.3 Performance Metrics and Results	11

5.4	Discussion	12
6	Conclusion and Future Work	14
	References	15

List of Figures

4.1	Offline phase of the proposed algorithm.	7
4.2	Online scheduling phase of the proposed algorithm.	9
5.1	Comparison of latency, quality score, and token count.	12

List of Tables

5.1	Comparison using 500, 1000, and 5000 prompts	13
-----	--	----

Chapter 1

Introduction

LLMs have achieved remarkable performance on a wide range of natural language processing tasks [1, 2, 3], including question answering, reasoning, summarizing, and conversational agents [4]. The rapid development of open-source LLMs has led to the availability of multiple LLMs with varying capabilities, computational requirements, and inference characteristics. Although some LLMs provide responses with a higher quality score, they often lead to higher latency and token count [5], whereas lighter LLMs offer lower latency at the cost of reduced response quality score. As a result, selecting an appropriate LLM for each query has become an important challenge in real-world deployments.

In most practical systems, a single LLM is selected and used for all queries, a static approach. However, this static LLM selection fails to account for LLMs' prompt-dependent behavior. LLM performance varies significantly depending on the structure, complexity, and domain of the input query [6, 7]. Consequently, an LLM that performs well for one type of prompt may perform poorly for another. This motivates the need for adaptive LLM selection algorithms [5] that can dynamically choose the most suitable LLM for each query while balancing multiple objectives such as latency, quality score, and token count. However, designing such adaptive selection algorithms introduces several challenges. First, accurately estimating the performance of different LLMs for unseen queries is important because of the variability in prompt semantics. Second, existing algorithms often rely on supervised learning or reinforcement learning techniques, which require labeled data, training overhead, and continuous maintenance. Third, querying multiple models or performing complex optimization can increase computational cost and latency, limiting scalability in real-time systems. These challenges highlight the need for lightweight and efficient scheduling strategies that can use existing information without adding significant computational overhead.

To address these challenges, SMS-LLM is proposed for adaptive multi-LLM selection. The central idea of SMS-LLM is that prompts with similar semantic characteristics may exhibit similar LLM performance patterns. To capture semantic similarity, a pretrained embedding model is used, which is inspired by techniques such as Sentence-BERT and SimCSE [8, 9]. Therefore, with the help of historical performance data (knowledge base) from previously observed prompts (prompt dataset), it is possible to estimate which LLM is likely to perform best for a new query. SMS-LLM constructs a knowledge base containing prompt embeddings along with per-LLM model performance metrics such as latency, quality score, and token count obtained via an LLM-as-a-Judge evaluation mechanism [10]. For an incoming prompt, its embedding is computed, and the most similar prompts are retrieved from the knowledge base. The performance metrics of these similar prompts are then used to guide LLM model selection.

The SMS-LLM algorithm is evaluated using three open-source LLMs: LLaMA 3, Mistral, and Qwen2.5 [11, 12, 13], deployed through the Ollama execution framework. Experiments are conducted on three different dataset sizes consisting of 500, 1000, and 5000 training prompts to analyze the scalability of the SMS-LLM algorithm. The SMS-LLM supports three scheduling objectives: latency-oriented scheduling (LS-LLM), quality-oriented scheduling (QS-LLM), and weighted multi-objective scheduling (SMS-LLM) that balances latency, quality score, and token count. Simulation results demonstrate that SMS-LLM consistently improves latency, quality, and token usage compared to static and random baselines. SMS-LLM reduces latency by up to 14.2%, improves quality score by up to 1.6%, and decreases token usage by up to 25.3% across different dataset sizes. The latency-oriented LS-LLM achieves up to 29.1% latency reduction, while QS-LLM improves the quality score by up to 5.76%. These results show that similarity-based scheduling enables efficient and scalable multi-objective LLM selection.

Chapter 2

Background

2.1 Large Language Models

Large Language Models (LLMs) are deep neural networks trained on large-scale text to perform a wide range of natural language processing tasks, including text generation, question answering, reasoning, and summarization. Modern LLMs use transformer models and many parameters to understand and produce human-like text.

Despite their strong capabilities, LLMs differ significantly in terms of model size, architecture, training data, and deployment configurations. As a result, their performance varies in multiple dimensions such as response quality, latency, and token count. Larger models often produce more accurate responses, but incur higher latency and cost, whereas smaller models offer faster responses at the expense of reduced quality. These trade-offs make it challenging to depend on a single model for all types of queries in real-world systems.

2.2 Prompt Embeddings and Semantic Similarity

To represent textual inputs in a form suitable for computation, embedding models are used to map the prompts to high-dimensional vector spaces. These embeddings capture semantic information so that similar texts are located close to each other in the embedding space. Semantic similarity plays an important role in many applications, including information retrieval and recommendation systems. In the context of LLMs, prompts with similar semantic characteristics often exhibit similar behavior in terms of model performance. This observation enables the use of embedding-based similarity to infer expected performance for new queries based on previously observed prompts.

2.3 Performance Metrics for LLMs

In practical deployments, LLM performance is evaluated using metrics such as latency, token count, and response quality. Latency measures the time taken to generate a response, which is important for real-time applications. Token count reflects the computational cost of processing and generating text. The quality of response indicates how well the output satisfies the query in terms of correctness, relevance, and clarity. Unlike latency and token count, quality is subjective and requires evaluation methods that approximate human judgment. Balancing these metrics is essential for efficient and scalable LLM systems.

2.4 LLM-as-a-Judge

Evaluating the quality of LLM-generated responses is challenging due to the need for a consistent and scalable assessment. Recent approaches use LLM themselves as evaluators, known as LLM-as-a-Judge, where a separate model assesses outputs based on criteria such as correctness, coherence, and instruction following. This method provides a scalable alternative to human evaluation and produces results that often align well with human judgments. It enables automated and consistent evaluation, making it suitable for large-scale and continuously evolving systems.

2.5 LLM Scheduling and Model Selection

In environments with multiple LLMs, selecting the most suitable model for each query is a key challenge. Since models differ in latency, quality, and cost, the selection process must balance multiple objectives. The goal of LLM scheduling is to assign each query to a model that best meets these objectives, such as minimizing latency, minimizing token count or maximizing quality.

However, estimating model performance for unseen queries is difficult due to variations in prompt semantics. Embedding-based similarity offers an effective solution using historical query information. By comparing a new query with similar past prompts, it is possible to estimate model performance and enable efficient, adaptive selection without requiring complex training or optimization.

Chapter 3

Related Work

Recent advancements in LLMs [11, 3] have demonstrated strong capabilities in reasoning, following instruction, and in-context learning. However, these models vary significantly in performance characteristics such as latency, quality, and cost, motivating the need for adaptive model selection strategies. Several works formulate LLM selection as a supervised learning problem. Maurya et al. [14] use a multi-label classifier to predict suitable models based on historical correctness labels. Similarly, routing-based approaches assign queries using learned policies or heuristics [15, 16], while Liu et al. [17] model selection as a multi-objective optimization problem. Reinforcement learning has also been explored, with Gupta et al. [18] proposing a hierarchical routing framework that learns scheduling policies through interaction. Although effective, these approaches require labeled data, training overhead, and accurate performance estimation, increasing system complexity.

Another line of work focuses on combining multiple LLMs. Wang et al. [19] propose mixture-based methods that aggregate outputs from multiple models to use complementary strengths. While these methods improve robustness and quality, they incur higher latency and cost due to multiple model queries. Accurate quality estimation is also critical in such systems. Recent work explores LLM-based evaluation methods, where LLMs act as judges to assess response quality [10, 20]. Structured approaches further improve alignment with human preferences using techniques such as chain-of-thought prompting and form-based scoring [21, 22]. These methods enable scalable evaluation without manual labeling.

Existing approaches rely on supervised training, reinforcement learning, or multi-model aggregation, which introduce additional overhead. In contrast, we propose a lightweight similarity-based scheduling approach that uses historical prompt embeddings and performance metrics (latency, quality score, and token count) for efficient multi-objective LLM selection without model training.

Chapter 4

Proposed Solution

This section introduces the proposed algorithm, SMS-LLM which optimizes latency, quality score and token count. Fig. 4.1 and Fig. 4.2 illustrate the overall workflow of SMS-LLM. The system operates in two phases: (i) an *offline* phase that constructs a knowledge base containing embeddings and per-LLM performance metrics for each prompt in the prompt dataset, and (ii) an *online* phase that assigns an incoming query to the most suitable LLM using similarity-based retrieval and estimated performance metrics.

4.0.1 Offline Phase: Knowledge Base Construction

In this phase, a knowledge base is constructed to store prompt embeddings along with per-LLM performance metrics. The embedding for each prompt in the dataset is computed using an embedding model. Subsequently, all prompts are executed on the available LLMs. During execution, the latency and token count for each model are recorded, and the generated responses are stored. Response quality is estimated using an LLM-as-a-Judge mechanism, which assigns a quality score to each response based on how well it follows the instruction, as well as correctness, clarity, and completeness. Finally, the prompt embeddings and corresponding per-LLM performance metrics are stored in the knowledge base. This process is illustrated in Fig. 4.1. The Algorithm 1 is used to construct the knowledge base.

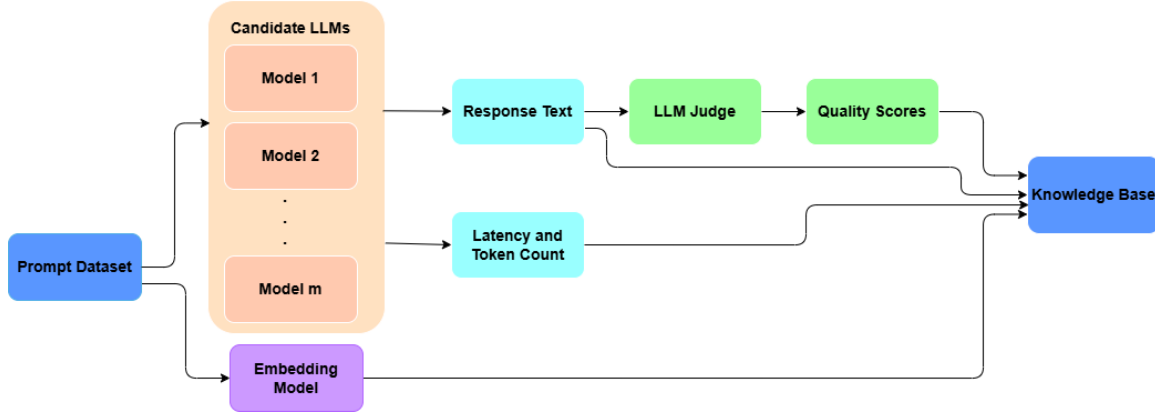


Figure 4.1: Offline phase of the proposed algorithm.

Algorithm 1 Knowledge Base Construction

Require: Prompt dataset D , model set M , embedding model E , judge model J

Ensure: Knowledge base $\mathcal{B}$

```

1: Initialize knowledge base  $\mathcal{B} \leftarrow \emptyset$ 
2: for each prompt  $p \in D$  do
3:   Compute embedding  $e_p = E(p)$ 
4:   for each model  $m \in M$  do
5:     Execute  $p$  on  $m$  and obtain response  $r_{m,p}$ 
6:     Measure latency  $L(m, p)$ 
7:     Measure token count  $T(m, p)$ 
8:   end for
9:   Evaluate responses  $\{r_{m,p} \mid m \in M\}$  using judge model  $J$ 
10:  Obtain quality scores  $\{Q(m, p) \mid m \in M\}$ 
11:  Store record  $(p, e_p, \{L(m, p)\}, \{T(m, p)\}, \{Q(m, p)\})$  in  $\mathcal{B}$ 
12: end for
13: return  $\mathcal{B}$ 
  
```

4.0.2 Online Phase: Similarity-Based Query Scheduling

In this phase, a new q_i is assigned to the most suitable LLM. When a q_i is provided to the scheduler, a scheduling objective is first selected from latency-oriented, quality-oriented, or weighted multi-objective scheduling. The latency-based scheduling (LS-LLM) algorithm selects the LLM with the lowest estimated latency. Mathematically,

$$m^* = \arg \min_{m \in M} \hat{L}(m \mid q) \quad (4.1)$$

The quality-based scheduling (QS-LLM) algorithm selects the LLM with the highest estimated quality score. Mathematically,

$$m^* = \arg \max_{m \in M} \hat{Q}(m \mid q). \quad (4.2)$$

In weighted multi-objective scheduling, the SMS-LLM selects the LLM that optimizes a weighted combination of latency, quality score, and token count. Mathematically,

$$m^* = \arg \max_{m \in M} (\alpha \cdot \tilde{Q}(m | q) - \beta \cdot \tilde{L}(m | q) - \gamma \cdot \tilde{T}(m | q)) \quad (4.3)$$

where $\tilde{Q}(\cdot)$, $\tilde{L}(\cdot)$, and $\tilde{T}(\cdot)$ denote normalized quality score, latency, and token count, respectively. α, β, γ control the trade-off between quality score, latency, and token count, respectively.

Once the objective is selected, the embedding e_{q_i} of q_i is computed. An embedding similarity function is then used to compare e_{q_i} with the prompt embeddings stored in the knowledge base, and prompts with similarity greater than the threshold τ are identified. From these, SMS-LLM selects the top K most similar prompts and estimates latency, token count, and quality score for each LLM using their performance metrics from the knowledge base.

If no sufficiently similar prompts are found, the system follows a fallback policy and assigns the query using global priors. For latency-oriented scheduling, the globally fastest LLM is selected, while for quality-oriented scheduling, the globally highest-quality LLM is selected. For weighted multi-objective scheduling, responses from both the globally fastest and globally highest-quality LLMs are generated and evaluated using the LLM-as-a-Judge mechanism, and the response with the better quality score is selected. In all cases, the generated responses are evaluated using the LLM-as-a-Judge mechanism, and their corresponding quality scores, including both responses in the weighted multi-objective case, are stored in the knowledge base along with latency and token count. This new prompt entry is then added to the knowledge base, enabling continual improvement over time. The online scheduling process is illustrated in Fig. 4.2. Algorithm 2 is used to schedule queries based on similarity.

Algorithm 2 Similarity-Based LLM scheduling

Require: Query q , LLM model pool M , knowledge base K

Ensure: Selected LLM model m^*

- 1: Compute query embedding $e_q = E(q)$
 - 2: Retrieve top- k most similar prompts S from K
 - 3: **if** $\max_similarity(S) \geq \tau$ **then**
 - 4: **for** each LLM model $m \in M$ **do**
 - 5: Estimate latency $\tilde{L}(m|q)$, token count $\tilde{T}(m|q)$ and quality score $\tilde{Q}(m|q)$
 - 6: **end for**
 - 7: Select LLM model m^* based on scheduling objective
 - 8: **else**
 - 9: Select fallback LLM model (global fastest or best-quality)
 - 10: **end if**
 - 11: Execute query q on LLM model m^*
 - 12: Evaluate response using LLM-as-a-Judge
 - 13: Update knowledge base K
 - 14: **return** m^*
-

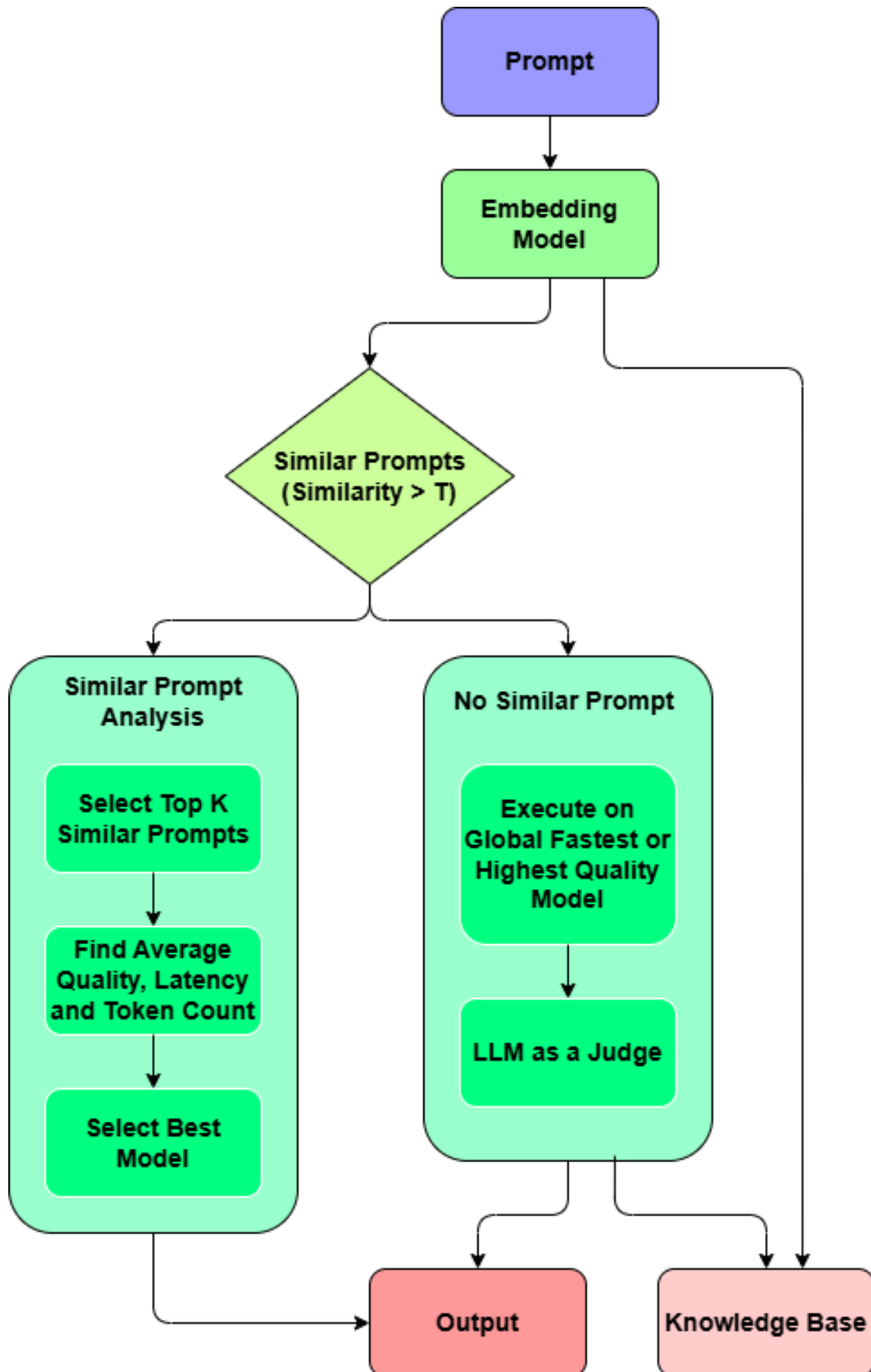


Figure 4.2: Online scheduling phase of the proposed algorithm.

Chapter 5

Simulation

5.1 Simulation Setup

To evaluate the effectiveness of SMS-LLM, experiments were conducted using prompts derived from the ShareGPT dataset, which contains diverse, multi-domain user prompts collected from real-world ChatGPT interactions. The algorithm was evaluated across three prompt dataset sizes: 500, 1000, and 5000 prompts. These prompts were used to construct the knowledge base during the offline phase. Prompt embeddings were generated using the *nomic-embed-text* embedding model. Each prompt was executed across three open-source LLMs: Llama3, Mistral, and Qwen2.5, deployed locally using Ollama, and the generated responses were evaluated using an LLM-as-a-Judge mechanism, in which responses were anonymized and evaluated by a judge LLM (Llama3). The judge assigns a score on a scale of 0 to 10 based on how well it follows the instruction, as well as correctness, clarity, and completeness. Scores are penalized when the response contains hallucinated or unsupported information.

For the online evaluation phase, a separate set of test prompts was used to assess scheduling performance. Due to hardware resource constraints, running multiple LLMs simultaneously was not feasible. Therefore, during the online evaluation phase, scheduling decisions were first made for all incoming prompts to determine the most suitable LLM for each prompt. After determining the scheduling assignments, prompts were grouped according to the selected LLM and executed in batches. This approach avoids frequent model switching and reduces the overhead associated with repeatedly loading different LLMs into memory. Consequently, the reported latency reflects the actual response generation time rather than additional delays caused by model loading. During inference, the algorithm recorded response latency and token count for each model execution.

5.2 Baseline Algorithms

To assess the effectiveness of LS-LLM, QS-LLM, and SMS-LLM, they are compared with several baseline algorithms. These baseline algorithms include single-model scheduling and random model scheduling. In the single-model baseline algorithms, all queries are executed on a fixed LLM, namely Llama3, Mistral, and Qwen2.5, and the corresponding latency, quality scores, and token count are recorded. These baseline algorithms represent static deployment scenarios in which a single LLM is used for all prompts, without adaptive scheduling. In addition, a Random LLM baseline is considered, in which each query is randomly assigned to one of the available LLMs. The performance of these baseline algorithms is compared with LS-LLM, QS-LLM, and SMS-LLM to evaluate the benefits of similarity-based scheduling.

5.3 Performance Metrics and Results

The experimental results demonstrate the effectiveness of SMS-LLM across varying dataset sizes. In addition, LS-LLM and QS-LLM are evaluated as objective-specific variants that emphasize latency and quality, respectively. LS-LLM achieves strong performance, particularly at larger scales, while SMS-LLM remains competitive with baseline algorithms. At 500 prompts, SMS-LLM records the lowest latency of 4440 ms, outperforming all baseline algorithms, including Qwen2.5 (4492 ms) and Random (4538 ms). At 1000 prompts, Qwen2.5 achieves the lowest latency of 4169 ms, followed by LS-LLM (4260 ms) and SMS-LLM (4493 ms). At the largest scale of 5000 prompts, LS-LLM significantly reduces latency to 5143 ms, improving over the worst baseline algorithm Qwen2.5 (7255 ms) by approximately 29.1% and outperforming Llama3 (6021 ms) by around 14.6%. SMS-LLM maintains competitive latency at 6227 ms, while simultaneously optimizing other objectives. QS-LLM consistently achieves the highest quality scores across dataset sizes, while SMS-LLM maintains competitive quality. At 500 prompts, QS-LLM achieves the highest quality score of 8.82, outperforming Mistral (8.78) and Llama3 (8.72). At 1000 prompts, QS-LLM and SMS-LLM both achieve the highest quality score of 8.73, exceeding all baseline algorithms. At 5000 prompts, QS-LLM again achieves the best quality score of 8.69, slightly surpassing Mistral (8.68), while SMS-LLM maintains competitive performance at 8.62, comparable to Random (8.62) and higher than Qwen2.5 (8.55). SMS-LLM demonstrates a balanced trade-off between token usage and performance. At 500 prompts, SMS-LLM reduces the token count to 500, compared to Qwen2.5 (627) and Random (523). At 1000 prompts, SMS-LLM records 510 tokens, remaining close to the best baseline

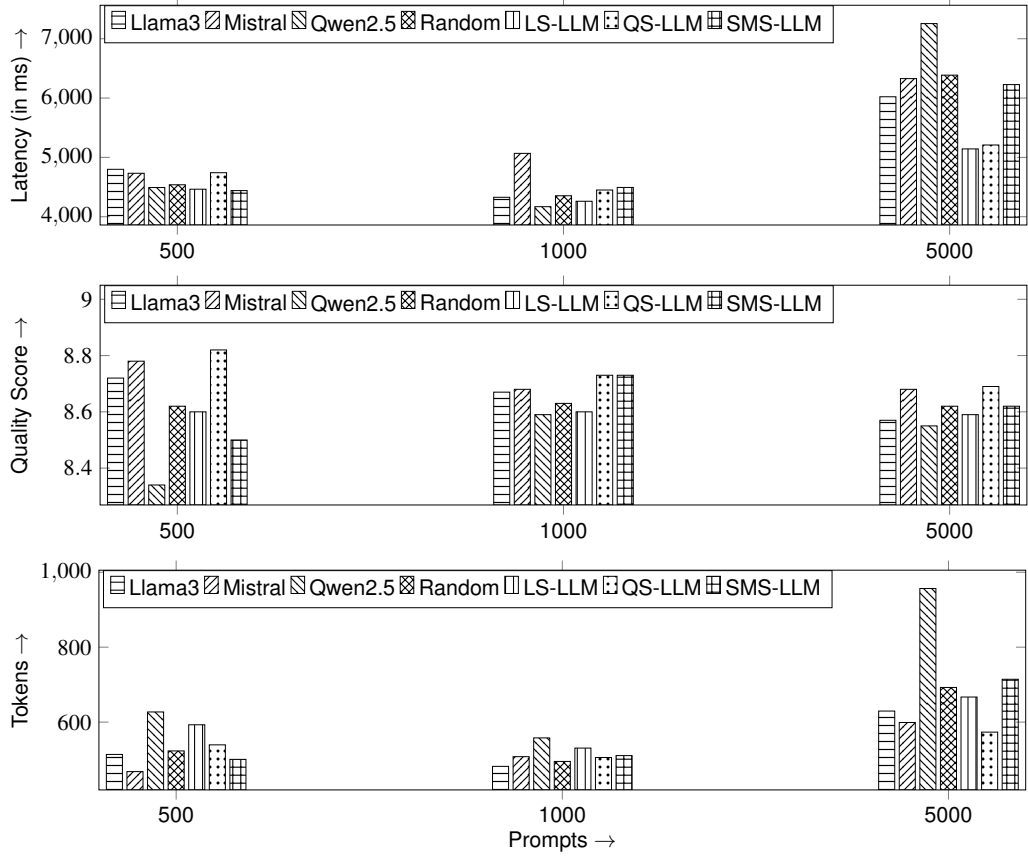


Figure 5.1: Comparison of latency, quality score, and token count.

algorithm Llama3 (481 tokens) while maintaining a higher quality score. At 5000 prompts, SMS-LLM has 714 token count, significantly lower than the worst baseline algorithm Qwen2.5 (956 tokens), achieving a reduction of approximately 25.3% while preserving balanced latency and quality score.

Overall, the results show that LS-LLM prioritizes latency minimization, QS-LLM emphasizes response quality, and SMS-LLM provides a balanced trade-off among latency, quality, and token count. SMS-LLM consistently achieves competitive performance across all objectives while avoiding the limitations of static algorithms, demonstrating the effectiveness of similarity-based scheduling.

5.4 Discussion

The improvements achieved by SMS-LLM arise from similarity-based scheduling, which assigns queries to LLMs using the historical performance of semantically

Table 5.1: Comparison using 500, 1000, and 5000 prompts

Algorithm	Latency			Quality Score			Token Count		
Prompts	500	1000	5000	500	1000	5000	500	1000	5000
Llama3	4799.10	4327.59	6021.88	8.72	8.67	8.57	513.74	481.89	629.48
Mistral	4732.28	5067.92	6328.07	8.78	8.68	8.68	467.90	507.76	598.98
Qwen2.5	4492.04	4169.23	7255.65	8.34	8.59	8.55	627.10	557.83	956.60
Random	4538.42	4353.21	6386.36	8.62	8.63	8.62	523.10	495.05	692.49
LS-LLM	4463.90	4260.57	5143.03	8.60	8.60	8.59	592.86	530.67	666.94
QS-LLM	4741.82	4450.15	5209.08	8.82	8.73	8.69	539.32	505.54	573.28
SMS-LLM	4440.56	4493.45	6227.23	8.50	8.73	8.62	500.56	510.60	714.35

similar prompts instead of relying on a fixed model. This enables objective-aware model selection and improves latency, quality, and token efficiency. LS-LLM shows the largest latency reductions since it explicitly prioritizes models with historically lower response time. At 500 prompts, LS-LLM reduces latency by 0.60–7.00% compared to baseline algorithms. The improvement increases to 1.50–15.90% at 1000 prompts and 14.60–29.10% at 5000 prompts, demonstrating its effectiveness at larger scales. QS-LLM improves the quality score by selecting models that have historically produced better responses for similar prompts. The quality improvement ranges from 0.46–05.76% at 500 prompts, 0.58–01.63% at 1000 prompts, and 0.12–1.63% at 5000 prompts. Although the absolute improvements are relatively small, they are consistent across dataset sizes, indicating stable quality optimization.

SMS-LLM demonstrates consistent improvements across multiple objectives by using similarity-based scheduling. At 500 prompts, SMS-LLM achieves latency improvements of 1.10–7.50% and reduces token count by 2.60–20.20% compared to baseline algorithms. At 1000 prompts, SMS-LLM improves quality score by 0.60–1.60% over baseline algorithms while maintaining competitive latency and token count. At 5000 prompts, SMS-LLM achieves latency reductions of up to 14.20% and token count reductions of up to 25.30%, while maintaining a comparable quality score. These results indicate that SMS-LLM effectively balances multiple objectives by dynamically selecting models based on the historical performance of semantically similar prompts.

Chapter 6

Conclusion and Future Work

This report presents SMS-LLM, a similarity-based multi-objective scheduling algorithm for adaptive multi-LLM selection. It uses semantic similarity between prompts and historical performance metrics to dynamically select the most suitable LLM for each query. By constructing a knowledge base containing prompt embeddings along with latency, quality score, and token count, SMS-LLM enables efficient model selection without requiring supervised training or reinforcement learning. It supports latency-oriented (LS-LLM), quality-oriented (QS-LLM), and weighted multi-objective scheduling, allowing flexible optimization based on the requirements. SMS-LLM is evaluated against static and random baseline algorithms, including LLama3, Mistral, Qwen2.5, and Random LLM selection algorithms. Simulation results demonstrate that similarity-based scheduling improves performance across multiple objectives. LS-LLM achieves latency reductions of up to 29.1%, while QS-LLM improves quality score upto 5.76%. The SMS-LLM provides balanced performance, achieving latency reductions of upto 14.2%, quality improvements of upto 1.6%, and token count reductions upto 25.3% across different dataset sizes. These improvements highlight that SMS-LLM provides an effective, lightweight, similarity-based scheduling algorithm that selects the most suitable LLM, enabling balanced optimization of latency, quality score, and token count without complex training. Future work can extend SMS-LLM by incorporating additional scheduling objectives, such as memory usage, and more comprehensive cost models beyond token count. Additionally, adaptive weight tuning for multi-objective scheduling and online updating of similarity thresholds may further improve performance and robustness in dynamic workloads.

References

- [1] Ashish Vaswani, Noam Shazeer, Niki Parmar, et al. Attention is all you need. *Advances in neural information processing systems*, 30, 2017. 1
- [2] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, et al. Palm: Scaling language modeling with pathways. *Journal of machine learning research*, 24(240):1–113, 2023. 1
- [3] Tom Brown, Benjamin Mann, Nick Ryder, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020. 1, 3
- [4] Sanjaya Kumar Panda, Sankalp Dubey, and Siba Mishra. Efficient task scheduling algorithms for decentralized large language model serving. In *TENCON 2025-2025 IEEE Region 10 Conference (TENCON)*, pages 1023–1027. IEEE, 2025. 1
- [5] Lingjiao Chen, Matei Zaharia, and James Zou. Frugalgpt: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023. 1
- [6] Kaijie Zhu, Jindong Wang, Jiaheng Zhou, et al. Promptrobust: Towards evaluating the robustness of large language models on adversarial prompts. In *Proceedings of the 1st ACM workshop on large AI systems and models with privacy and safety analysis*, pages 57–68, 2023. 1
- [7] Ali Borji. A categorical archive of chatgpt failures. *arXiv preprint arXiv:2302.03494*, 2023. 1
- [8] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*, pages 3982–3992, 2019. 1

- [9] Tianyu Gao, Xingcheng Yao, and Danqi Chen. Simcse: Simple contrastive learning of sentence embeddings, 2022. 1
- [10] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems*, 36:46595–46623, 2023. 1, 3
- [11] Hugo Touvron, Thibaut Lavril, Gautier Izacard, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023. 1, 3
- [12] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, et al. Mistral 7b, 2023. 1
- [13] Binyuan Hui, Jian Yang, Zeyu Cui, et al. Qwen2. 5-coder technical report. *arXiv preprint arXiv:2409.12186*, 2024. 1
- [14] Kaushal Kumar Maurya, KV Aditya Srivatsa, and Ekaterina Kochmar. Selectllm: Query-aware efficient selection algorithm for large language models. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 20847–20863, 2025. 3
- [15] Cheng Qian, Zuxin Liu, Shirley Kokane, et al. xrouter: Training cost-aware llms orchestration system via reinforcement learning. *arXiv preprint arXiv:2510.08439*, 2025. 3
- [16] Yueyue Liu, Hongyu Zhang, and Yuantian Miao. Smartllms scheduler: A framework for cost-effective llms utilization. *arXiv preprint arXiv:2508.03258*, 2025. 3
- [17] Yueyue Liu, Hongyu Zhang, Yuantian Miao, Van-Hoang Le, and Zhiqiang Li. Optllm: Optimal assignment of queries to large language models. In *2024 IEEE International Conference on Web Services (ICWS)*, pages 788–798. IEEE, 2024. 3
- [18] Nikunj Gupta, Bill Guo, Rajgopal Kannan, and Viktor K Prasanna. Hierrouter: Coordinated routing of specialized large language models via reinforcement learning. *arXiv preprint arXiv:2511.09873*, 2025. 3
- [19] Xinyuan Wang, Yanchi Liu, Wei Cheng, et al. Mixllm: Dynamic routing in mixed large language models. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 10912–10922, 2025. 3

- [20] Cheng-Han Chiang and Hung-yi Lee. Can large language models be an alternative to human evaluations? In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15607–15631, 2023. 3
- [21] Yang Liu, Dan Iter, Yichong Xu, et al. G-eval: Nlg evaluation using gpt-4 with better human alignment. In *Proceedings of the 2023 conference on empirical methods in natural language processing*, pages 2511–2522, 2023. 3
- [22] Helia Hashemi, Jason Eisner, Corby Rosset, et al. Llm-rubric: A multidimensional, calibrated approach to automated evaluation of natural language texts. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13806–13834, 2024. 3